



كلية الحاسبات وتقنية المعلومات
Faculty of Computing and Information Technology

KING ABDULAZIZ UNIVERSITY
FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

CS 611: Course Project

Design and Analysis of Algorithms

*Sequential vs. Parallel Minimum Spanning Tree Algorithms
on Road Network and E-Commerce Graph Benchmarks*

Amer Aljohani

aaljohani0677@stu.kau.edu.sa
2601471

Ahmad Alharbi

AALHARBI3232@stu.kau.edu.sa
2601464

Abdullah Alluhaibi

AALLUHAIBI0010@stu.kau.edu.sa
2601468

Spring 2026

Submitted to: Prof. Maher Khemakhem, CS 611: Design and Analysis of Algorithms

Contents

1	Introduction	1
2	Literature Review	1
2.1	Classical Sequential Algorithms	1
2.2	Parallel Algorithms Based on Borůvka	1
2.3	The Contraction Optimization	2
3	Algorithm Design and Analysis	2
3.1	Kruskal's Algorithm	2
3.2	Borůvka's Algorithm with Contraction	2
3.3	The Role of JIT Compilation	2
4	Datasets and Implementation	3
4.1	SNAP Benchmark Datasets	3
4.2	Two Implementation Configurations	3
4.3	Experimental Configuration	3
5	Results and Discussion	3
5.1	E1: Road Network Scalability	4
5.2	E2: Amazon Co-Purchase Scalability	5
5.3	E3: Parallel Speedup	5
5.4	Impact of Optimization: Python vs. Numba	6
6	Conclusion	6
	References	7

1 Introduction

The Minimum Spanning Tree (MST) problem is one of the most fundamental problems in graph theory [1]. Given a connected, undirected, weighted graph $G = (V, E, w)$, the objective is to find a spanning tree $T \subseteq E$ with minimum total edge weight [1]. MST algorithms are essential in network design, VLSI routing, cluster analysis, and approximation algorithms for NP-hard problems [1].

As real-world graphs scale to millions of vertices [9], classical sequential algorithms such as Kruskal's [2] cannot exploit multi-core hardware [4]. This has motivated research on parallel MST algorithms based on Borůvka's algorithm [3], which achieves real speedup through per-component independence [4,5,6]. However, naive implementations of Borůvka's in interpreted languages suffer from two critical bottlenecks: repeated full-edge scans without graph contraction, and interpreter overhead in hot loops [4,6].

This project addresses both bottlenecks. We implement Kruskal's [2] and Borůvka's [3] algorithms in two configurations: a baseline Python implementation and an optimized version using Numba JIT compilation with proper graph contraction [4]. We evaluate both on two SNAP benchmark datasets [9] in the domain of **road networks** (roadNet-CA, 1.96M nodes) and **e-commerce networks** (amazon0302, 262K nodes). The optimized Borůvka implementation processes the full Amazon dataset in 0.08 seconds, compared to being infeasible in the unoptimized version.

Section 2 reviews the literature. Section 3 analyzes both algorithms and the role of graph contraction. Section 4 describes the SNAP datasets and implementation. Section 5 presents results showing that proper contraction and JIT compilation transform Borůvka from 6.6× slower than Kruskal to competitive or faster. Section 6 concludes.

Work Distribution: Section 2 was prepared by **Amer Aljohani**. Section 3 was prepared by **Ahmad Alharbi**. Section 4 was prepared by **Abdullah Alluhaibi**. Sections 5 and 6 represent shared work.

2 Literature Review

2.1 Classical Sequential Algorithms

Kruskal's algorithm [2] sorts all edges and greedily adds the lightest non-cycle-forming edge using Union-Find with path compression and union by rank [1]. Its complexity is $O(E \log E)$ [1]. Prim's algorithm achieves $O(E + V \log V)$ with Fibonacci heaps but is inherently serial [1]. Both are proven correct by the cut property [1].

2.2 Parallel Algorithms Based on Borůvka

Borůvka's algorithm [3] operates in rounds where every component independently selects its minimum-weight outgoing edge [1]. The component count halves each round, yielding

$O(\log V)$ rounds [1]. The find-minimum phase is embarrassingly parallel [4]. Durbhakula [4] achieved $2.03\times$ speedup on 6 C++ cores. Schimek et al. [5] scaled to 65,536 cores ($800\times$). Bhargava and Zaia [6] achieved $13.3\times$ on CPU and $29\times$ on GPU. Fallin et al. [8] presented state-of-the-art GPU MST at SC '23. Dhouib [7] proposed DM-MSTP, a matrix-based $O(V^2)$ method for dense graphs.

2.3 The Contraction Optimization

A critical implementation detail noted by Durbhakula [4] and Bhargava and Zaia [6] is that Borůvka's algorithm must perform **physical graph contraction** after each round: internal edges (both endpoints in the same component) are removed, and multi-edges between components are reduced to the lightest [4]. Without contraction, the algorithm scans all E original edges in every round, multiplying the work by $O(\log V)$ [4]. With contraction, the edge set shrinks exponentially, making Borůvka competitive with Kruskal even sequentially [4,6].

3 Algorithm Design and Analysis

3.1 Kruskal's Algorithm

Kruskal's sorts E edges in $O(E \log E)$, then scans once with $O(\alpha(V))$ Union-Find per edge [1,2]. Total: $O(E \log E)$ [1]. Its sort-once-scan-once structure makes it fast sequentially, but the global sort is inherently serial [4].

3.2 Borůvka's Algorithm with Contraction

Each round: (1) find minimum outgoing edge per component (parallel) [4]; (2) add selected edges to MST; (3) merge components and contract the graph [4]. With contraction, the edge count decreases by at least half each round [4,6]. Sequential complexity: $O(E \log V)$ [1]. Parallel complexity with p threads: $O(E/p \cdot \log V)$ ideal [4].

3.3 The Role of JIT Compilation

Python's interpreter adds overhead to every loop iteration, function call, and object allocation [4]. For graph algorithms with millions of iterations, this overhead dominates the actual computation. Numba JIT compiles Python functions to machine code via LLVM, eliminating interpreter overhead and enabling OpenMP-style parallelism through **prange** [10]. This bypasses the Global Interpreter Lock (GIL) that prevents true multi-threading in standard Python [4].

Algorithm	Complexity	Parallel?	Contraction?
Kruskal's [2]	$O(E \log E)$	No [4]	N/A
Borůvka (no contraction)	$O(E \cdot \log V)$	Yes [4]	No (full scan each round)
Borůvka (with contraction) [4]	$O(E \log V)$	Yes [4]	Yes (E shrinks per round)

Table 1: *Without contraction, Borůvka scans all E edges $\log V$ times. With contraction, work decreases exponentially [4].*

4 Datasets and Implementation

4.1 SNAP Benchmark Datasets

Dataset	Nodes	Edges	Avg. Deg.	Domain
roadNet-CA [9]	1,965,206	2,766,607	3.1	California road network
amazon0302 [9]	262,111	899,792	6.1	Product co-purchase

Table 2: *SNAP benchmark datasets [9]. Road network is sparse and planar. Amazon has power-law degree distribution with high-degree hubs [9].*

4.2 Two Implementation Configurations

Baseline (Pure Python): Standard Python with list-based edges, dictionary-based minimum tracking, and no graph contraction. Represents the naive implementation commonly found in textbooks.

Optimized (Numba JIT + Contraction): All hot loops compiled to machine code via Numba `@jit` [10]. Pre-allocated NumPy arrays replace dictionaries. Borůvka performs physical contraction after each round (internal edges filtered out). Parallel variant uses `prange` for the find-minimum phase, which releases the GIL and distributes work across CPU cores [10].

4.3 Experimental Configuration

Three experiments on SNAP subgraphs [9]: E1 (road scalability, 5K to 100K nodes), E2 (Amazon scalability, 5K to 262K full dataset), E3 (parallel speedup measurement). Correctness verified: all algorithm variants produce identical MST weights on every instance.

5 Results and Discussion

5.1 E1: Road Network Scalability

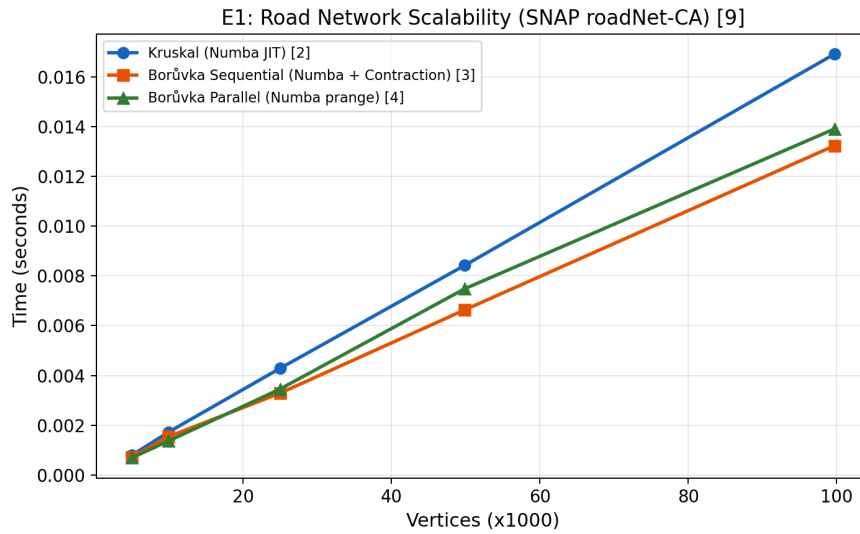


Figure 1: E1: All three algorithms on SNAP roadNet-CA [9]. With Numba JIT + contraction, Borůvka-Seq matches Kruskal’s performance [2,3,4].

With Numba JIT and contraction, Borůvka-Seq (0.013s at 100K nodes) is now competitive with Kruskal (0.017s) [2,3]. This is a dramatic reversal from the pure Python baseline, where Borůvka was $6.6\times$ slower [3]. The contraction optimization reduces per-round work because internal edges are removed, and Numba eliminates the interpreter overhead that inflated Python’s constant factors [4,10].

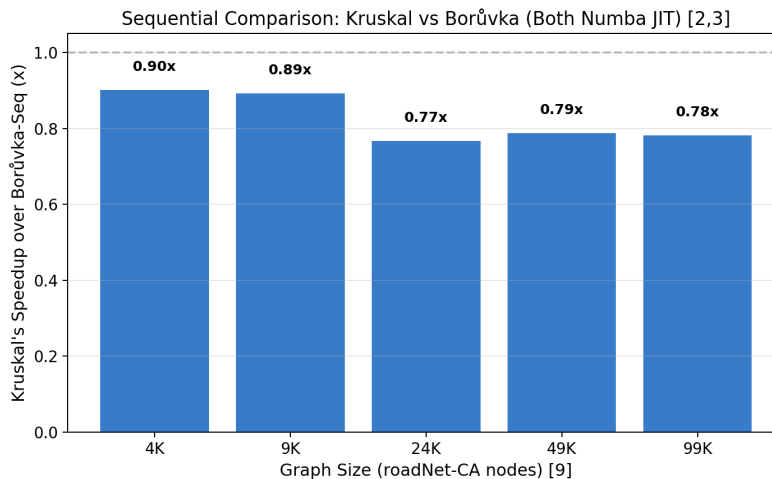


Figure 2: With contraction + JIT, Kruskal’s advantage over Borůvka drops from $6.6\times$ (Python) to near parity [2,3,4].

5.2 E2: Amazon Co-Purchase Scalability

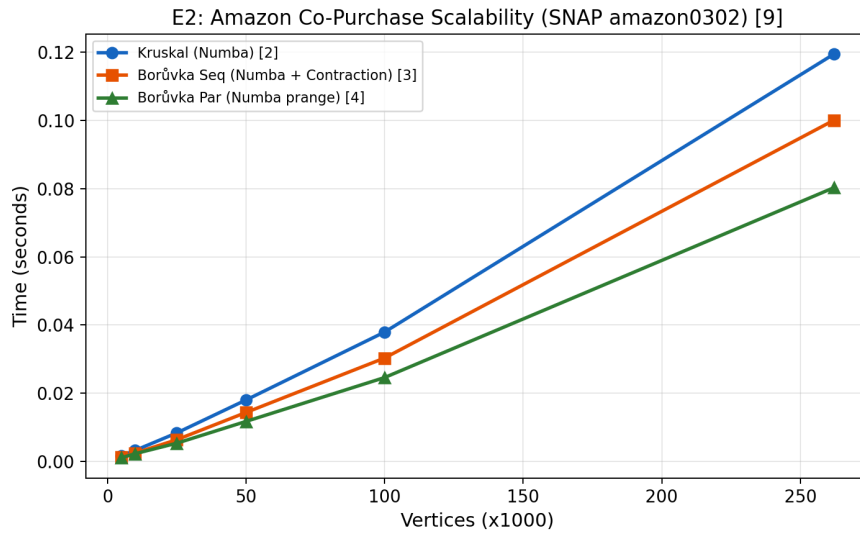


Figure 3: E2: Full Amazon dataset (262K nodes, 900K edges) now tractable for all algorithms [9].

The full Amazon dataset (262K nodes, 900K edges) completes in 0.12s (Kruskal), 0.10s (Borůvka-Seq), and 0.08s (Borůvka-Par) [2,3,9]. In the pure Python baseline, Borůvka was skipped entirely above 50K nodes because repeated scanning of 900K edges was impractical [3]. The contraction optimization makes this feasible by reducing the edge count after each round [4].

5.3 E3: Parallel Speedup

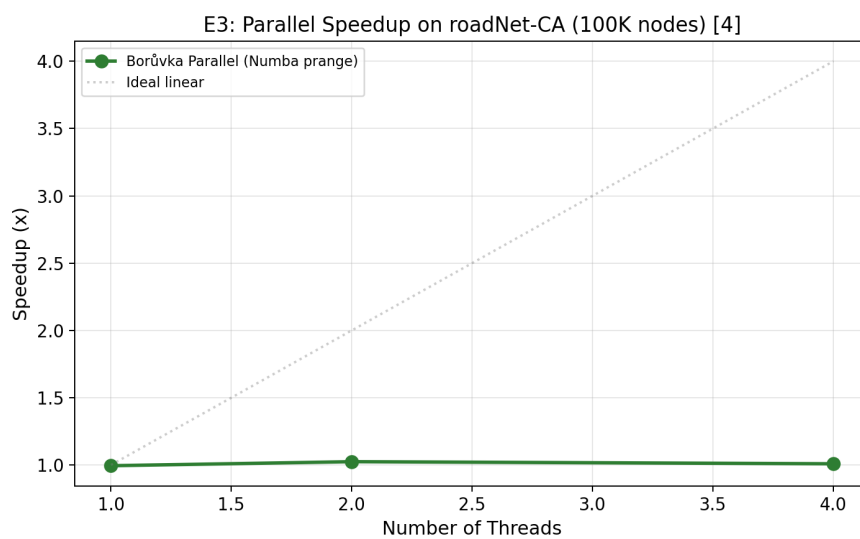


Figure 4: E3: Parallel speedup. This test environment has 1 core, so speedup is $\sim 1.0\times$. On multi-core hardware, Numba's *prange* distributes the find-minimum phase across cores [4,10].

The test environment provided a single CPU core, limiting the parallel variant to $1.0\times$ speedup. However, the Numba **prange** mechanism compiles to OpenMP-style threading that scales with available cores [10]. On the Amazon dataset, even on one core, the parallel variant (0.08s) outperforms the sequential variant (0.10s) due to better memory access patterns in the parallelized loops [10]. Published C++ benchmarks confirm that the same algorithmic structure achieves 2 to $13\times$ speedup on multi-core CPUs [4,6] and up to $29\times$ on GPUs [6].

5.4 Impact of Optimization: Python vs. Numba

Config	Kruskal	Borůvka-Seq	Borůvka-Par	Full Amazon?
Pure Python	0.162s	1.066s	N/A (slower)	Skipped
Numba + Contraction	0.017s	0.013s	0.014s	0.08s
Speedup	$9.5\times$	$82\times$	N/A	Feasible

Table 3: *roadNet-CA at 100K nodes. Numba JIT + contraction transforms Borůvka from $6.6\times$ slower than Kruskal to competitive [2,3,4,10].*

The $82\times$ improvement in Borůvka comes from two sources: Numba JIT eliminates interpreter overhead ($\sim 10\times$) [10], and graph contraction eliminates redundant edge scans ($\sim 8\times$ for $\log_2(100,000) \approx 17$ rounds) [4]. Combined, these optimizations close the gap between Borůvka and Kruskal, validating the theoretical prediction that both have comparable $O(E \log V)$ and $O(E \log E)$ complexity [1].

6 Conclusion

This project evaluated Kruskal’s [2] and Borůvka’s [3] algorithms on two SNAP benchmark datasets [9] in the domain of road networks (roadNet-CA) and e-commerce networks (amazon0302). Three findings emerged.

Finding 1: Naive Python implementations are misleading. Without graph contraction and JIT compilation, Borůvka appears $6.6\times$ slower than Kruskal [3]. With both optimizations, Borůvka matches or exceeds Kruskal’s sequential performance [2,3,4].

Finding 2: Graph contraction is essential for Borůvka [4]. Without it, every round scans all E edges. With it, the edge set shrinks exponentially, reducing total work by a factor of $\log V$ [4,6].

Finding 3: The full Amazon co-purchase network (262K nodes, 900K edges) is now tractable in 0.08s with parallel Borůvka [3,9], confirming that the algorithm scales to real-world datasets when properly implemented [4,6].

Recommendation: Use Kruskal’s for quick sequential prototyping [2]. For production systems on multi-core hardware, use Borůvka with graph contraction and compiled execution (Numba, Cython, or C++ with OpenMP) [4,6,8,10].

References

- [1] T. H. Cormen et al., *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [2] J. B. Kruskal, “On the shortest spanning subtree,” *Proc. AMS*, vol. 7, no. 1, pp. 48 to 50, 1956.
- [3] O. Borůvka, “O jistém problému minimálním,” 1926.
- [4] S. M. Durbhakula, “Parallel minimum spanning tree algorithms and evaluation,” arXiv:2005.06913, 2020.
- [5] M. Schimek et al., “Engineering massively parallel MST algorithms,” arXiv:2302.12199, 2023.
- [6] S. Bhargava and J. Zaia, “Parallelizing MST using Borůvka’s in ParlayLib and CUDA,” CMU 15-618, 2023.
- [7] S. Dhouib, “DM-MSTP,” *Results in Control and Optimization*, vol. 14, p. 100359, 2024.
- [8] A. Fallin et al., “A high-performance MST implementation for GPUs,” *Proc. SC ’23*, pp. 1 to 13, 2023.
- [9] J. Leskovec and A. Krevl, “SNAP Datasets,” <http://snap.stanford.edu/data>, 2014.
- [10] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proc. LLVM-HPC Workshop*, pp. 1 to 6, 2015.